IDK - Game Documentation

Contents

1. Project Overview 3
2. Technical Architecture 3
   2.1 File Structure 3
   2.2 Core Components & Interactions 4
   2.3 Libraries and Frameworks 5
3. Codebase Documentation 5
   BootScene.js 5
   GameManagerScene.js 6
   MainScene.js 6
   BuildModeScene.js 7
   ShopMenuScene.js 7
   InventoryScene.js 8
   SellMenuScene.js 8
   UIScene.js 9

1. Project Overview

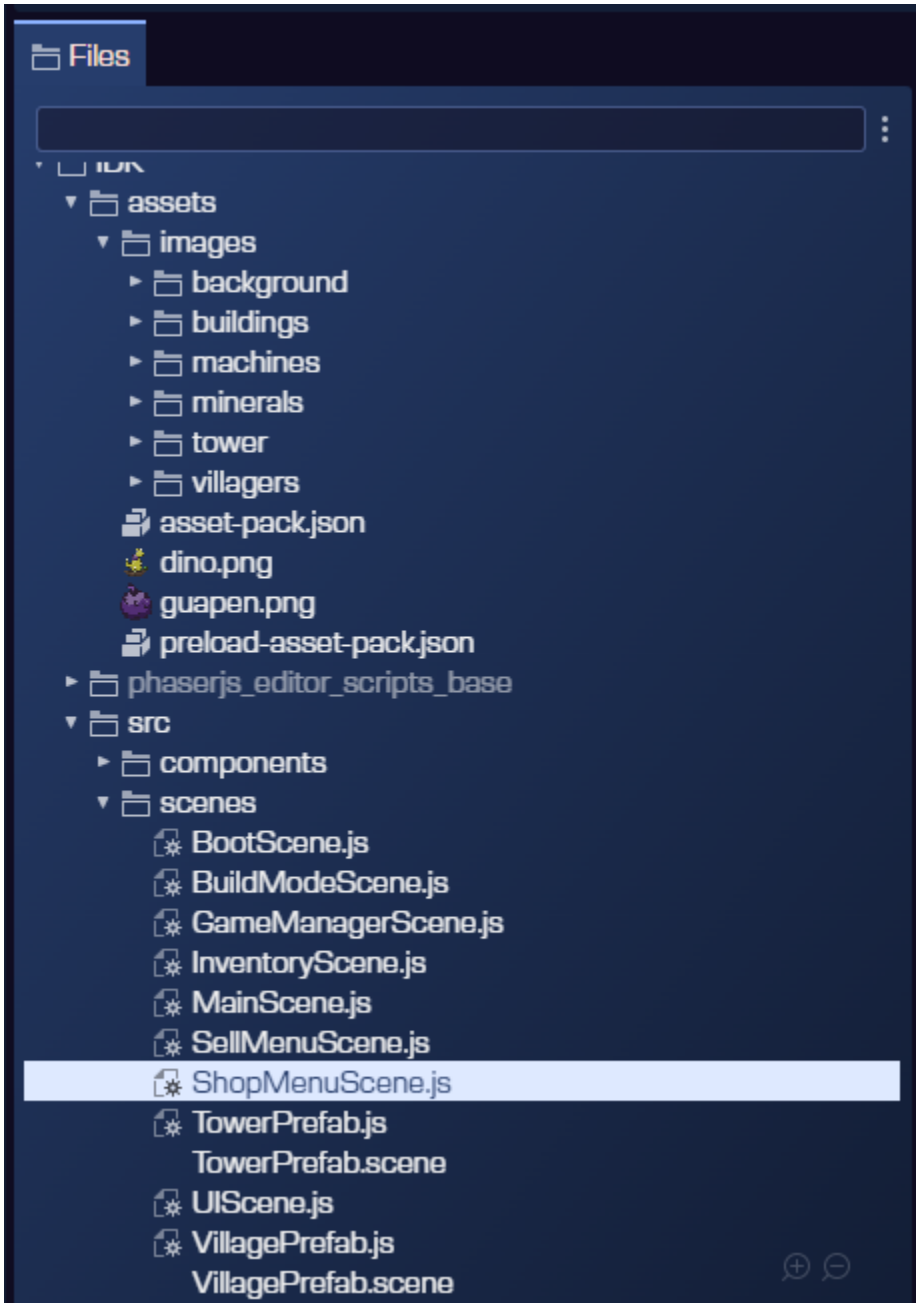
IDK (Idle Development Kingdom) is a 2D resource management and production game developed using Phaser 4. Players build machines to produce materials, sell resources to villagers, manage day-night cycles, upgrade their production capabilities, and ultimately unlock the final game-ending upgrade.

2. Technical Architecture

2.1 File Structure

/assets/images/ - Contains background tiles, building sprites, tower image, machine icons, mineral icons, and villager portraits.

/Scenes/ - Contains all JavaScript logic files for the game's different scenes.



2.2 Core Components & Interactions

The game is divided into several scenes managing different systems:

- **BootScene**: Preloads assets and launches the game.
- **GameManagerScene**: Handles production logic, day-night cycles.
- **MainScene**: Displays world, manages player interactions.
- **BuildModeScene**: Grid system to place production machines.
- **ShopMenuScene**: Purchase new machines.
- **InventoryScene**: Shows produced resources.
- **SellMenuScene**: Sell resources to villagers.
- **UIScene**: Upgrade tree and game-ending system.

2.3 Libraries and Frameworks

The project is built using Phaser 4 for game logic and rendering. No external libraries were used beyond core Phaser.

3. Codebase Documentation

BootScene.js

Preloads all assets (backgrounds, machines, minerals, villagers) and starts the game.

```
export default class BootScene extends Phaser.Scene {
    constructor() {
        super('BootScene');
    }

    preload() {
        // Backgrounds
        this.load.image('land_1', 'assets/images/background/land_1.png');
        this.load.image('land_2', 'assets/images/background/land_2.png');

        // Buildings
        for (let i = 1; i <= 6; i++) {
            this.load.image(`building_${i}`, `assets/images/buildings/building_${i}.png`);
        }

        this.load.image('PlayerTower', 'assets/images/tower/PlayerTower.png');

        // Machine Images by Product
        const machineProducts = ['iron', 'copper', 'steel', 'amethyst', 'emerald', 'magmastone', 'quartz', 'titanium'];
        machineProducts.forEach(name => {
            this.load.image(`machine_${name}`, `assets/images/machines/machine_${name}.png`);
        });

        // Mineral Icons
        const minerals = ['iron', 'copper', 'steel', 'amethyst', 'emerald', 'magmastone', 'quartz', 'titanium'];
        minerals.forEach(name => {
            this.load.image(name, `assets/images/minerals/${name}.png`);
        });

        // Villager Portraits
        const villagers = ['aeron', 'branna', 'cael', 'eira', 'lorcan', 'saoirse'];
        villagers.forEach(name => {
            this.load.image(`villager_${name}`, `assets/images/villagers/${name}.png`);
        });
    }

    create() {
        this.scene.launch('GameManagerScene');
        this.scene.start('MainScene');
    }
```

GameManagerScene.js

Runs production cycles, applies upgrades, handles day-night transitions, manages global registry.

```
export default class GameManagerScene extends Phaser.Scene {
    constructor() {
        super({ key: 'GameManagerScene', active: true });
    }

    init() {
        this.isNight = this.registry.get('isNight') || false;
        this.timeLeft = 15000;

        // Init registry defaults
        if (!this.registry.has('money')) this.registry.set('money', 100);
        if (!this.registry.has('inventory')) this.registry.set('inventory', []);
        if (!this.registry.has('placedMachines')) this.registry.set('placedMachines', []);

        // Generate random requests for villagers
        if (!this.registry.has('villageRequests')) {
            const items = ['iron', 'copper', 'steel', 'amethyst', 'emerald', 'magmastone', 'quartz', 'titanium'];
            this.registry.set('villageRequests', Array.from({ length: 6 }, () =>
                Phaser.Utils.Array.GetRandom(items)
            ));
        }
    }

    create() {
        // Machine production tick every second
        this.time.addEvent({
            delay: 1000,
            loop: true,
            callback: () => this.runMachineProduction()
        });

        // Day/night switch
        this.time.addEvent({
            delay: this.timeLeft,
            loop: true,
            callback: () => {
                this.isNight = !this.isNight;
                this.registry.set('isNight', this.isNight);

                // New requests at night
```

MainScene.js

Main gameplay scene. Displays world, lets player access shop, build mode, inventory, and tower upgrades.

```
export default class MainScene extends Phaser.Scene {
    constructor() {
        super('MainScene');
    }

    init(data) {
        // Load or set up game state
        this.placedMachines = data.machines || this.registry.get('placedMachines') || [];
        this.playerInventory = data.inventory || this.registry.get('inventory') || [];
        this.money = data.money !== undefined ? data.money : this.registry.get('money') || 100;
        this.isNight = data.isNight ?? this.registry.get('isNight') ?? false;
        this.villageRequests = data.villageRequests || this.registry.get('villageRequests') || [];
    }

    create() {
        // Place background
        for (let y = 0; y < 2160; y += 256) {
            for (let x = 0; x < 3840; x += 256) {
                this.add.image(x, y, 'land_1').setOrigin(0);
            }
        }
        for (let i = 0; i < 30; i++) {
            const x = Phaser.Math.Between(0, 3840);
            const y = Phaser.Math.Between(0, 2160);
            this.add.image(x, y, 'land_2').setOrigin(0.5);
        }

        // Tower setup
        this.tower = this.add.image(300, 540, 'PlayerTower')
            .setOrigin(0.5)
            .setInteractive({ useHandCursor: true });

        this.tower.on('pointerdown', () => {
            this.scene.launch('UIScene');
            this.scene.pause();
        });

        // Village houses with random requests
        const buildingPositions = [
            { x: 1300, y: 600 },
```

BuildModeScene.js

Grid-based machine placement system with dynamic UI. Supports inventory selection and building logic.

```javascript
export default class BuildModeScene extends Phaser.Scene {
    constructor() {
        super('BuildModeScene');
    }

    init(data) {
        // Load player data
        this.placedMachines = data.machines || this.registry.get('placedMachines') || [];
        this.playerInventory = data.playerInventory || this.registry.get('inventory') || [];
        this.money = data.money ?? this.registry.get('money') ?? 100;
        this.isNight = data.isNight ?? this.registry.get('isNight') ?? false;
        this.timeLeft = data.timeLeft ?? this.registry.get('timeLeft') ?? 15000;
        this.villageRequests = data.villageRequests || this.registry.get('villageRequests') || [];

        // Grid and selection setup
        this.selectedMachine = null;
        this.gridStartX = 600;
        this.gridStartY = 400;
        this.gridCols = 5;
        this.gridRows = 3;
        this.cellSize = 128;
        this.grid = [];
        this.machineUI = [];
    }

    create() {
        // Background
        for (let y = 0; y < 2160; y += 256) {
            for (let x = 0; x < 3840; x += 256) {
                this.add.image(x, y, 'land_1').setOrigin(0);
            }
        }
        for (let i = 0; i < 20; i++) {
            const x = Phaser.Math.Between(0, 3840);
            const y = Phaser.Math.Between(0, 2160);
            this.add.image(x, y, 'land_2').setOrigin(0.5);
        }

        this.createBuildGrid();

        // Existing machines
```

ShopMenuScene.js

Shop menu where players buy production machines with in-game money.

```javascript
export default class ShopMenuScene extends Phaser.Scene {
    constructor() {
        super('ShopMenuScene');
    }

    init(data) {
        // Load player data
        this.playerInventory = this.registry.get('inventory') || [];
        this.placedMachines = data.placedMachines || this.registry.get('placedMachines') || [];
        this.money = data.money ?? this.registry.get('money') ?? 0;
        this.isNight = data.isNight ?? this.registry.get('isNight') ?? false;
        this.timeLeft = data.timeLeft ?? this.registry.get('timeLeft') ?? 15000;
        this.villageRequests = data.villageRequests || this.registry.get('villageRequests') || [];
    }

    create() {
        // Available machines to purchase
        const machinesForSale = [
            { name: 'Iron Machine', texture: 'machine_iron', produces: 'iron', price: 50 },
            { name: 'Copper Machine', texture: 'machine_copper', produces: 'copper', price: 75 },
            { name: 'Steel Machine', texture: 'machine_steel', produces: 'steel', price: 100 },
            { name: 'Amethyst Machine', texture: 'machine_amethyst', produces: 'amethyst', price: 200 },
            { name: 'Emerald Machine', texture: 'machine_emerald', produces: 'emerald', price: 300 },
            { name: 'MagmaStone Machine', texture: 'machine_magmastone', produces: 'magmastone', price: 450 },
            { name: 'Quartz Machine', texture: 'machine_quartz', produces: 'quartz', price: 700 },
            { name: 'Titanium Machine', texture: 'machine_titanium', produces: 'titanium', price: 1000 }
        ];

        // Shop panel
        this.add.rectangle(960, 540, 900, 800, 0x333333, 0.95).setStrokeStyle(4, 0xffffff, 0.5);

        // Shop title
        this.add.text(720, 180, 'Shop - Buy Machines', {
            fontSize: '36px',
            color: '#ffffff',
            fontFamily: 'Arial'
        });

        // Money display
        this.moneyText = this.add.text(720, 130, `$${this.money}`, {
```

InventoryScene.js

Displays inventory contents with dynamic scaling based on number of items.

```javascript
export default class InventoryScene extends Phaser.Scene {
    constructor() {
        super('InventoryScene');
    }

    init(data) {
        // Load saved or incoming data
        this.playerInventory = data.inventory || this.registry.get('inventory') || [];
        this.placedMachines = data.machines || this.registry.get('placedMachines') || [];
        this.money = data.money ?? this.registry.get('money') ?? 0;
        this.isNight = data.isNight ?? this.registry.get('isNight') ?? false;
        this.villageRequests = data.villageRequests || this.registry.get('villageRequests') || [];
        this.uiElements = [];  // Dynamic UI cleanup list
    }

    create() {
        // Count each item type in inventory
        const itemCounts = {};
        this.playerInventory.forEach(item => {
            const key = item.produces || item;
            itemCounts[key] = (itemCounts[key] || 0) + 1;
        });

        // Inventory box resizes based on item count
        const visibleItems = Object.keys(itemCounts).length;
        const minHeight = 400;
        const dynamicHeight = 200 + visibleItems * 60;
        const panelHeight = Math.max(minHeight, dynamicHeight);

        const panel = this.add.rectangle(960, 540, 600, panelHeight, 0x2a2a2a, 0.95);
        panel.setStrokeStyle(4, 0xffffff, 0.6);

        this.add.text(780, 300, 'Your Inventory', {
            fontSize: '36px',
            color: '#ffffff',
            fontFamily: 'Arial',
            stroke: '#000',
            strokeThickness: 2
        });
```

SellMenuScene.js

Allows the player sell requested items to villagers, apply bonus upgrades if unlocked.

```javascript
export default class SellMenuScene extends Phaser.Scene {
    constructor() {
        super('SellMenuScene');
    }

    init(data) {
        // Basic data handling
        this.inventory = data.inventory || this.registry.get('inventory') || [];
        this.money = data.money ?? this.registry.get('money') ?? 0;
        this.placedMachines = data.machines || this.registry.get('placedMachines') || [];
        this.isNight = data.isNight ?? this.registry.get('isNight') ?? false;
        this.timeLeft = data.timeLeft ?? this.registry.get('timeLeft') ?? 15000;
        this.buildingIndex = data.buildingIndex ?? 0;

        this.villageRequests = this.registry.get('villageRequests') || [];
        this.wants = this.villageRequests[this.buildingIndex] || 'iron';

        this.villagerNames = ['Aeron', 'Branna', 'Cael', 'Eira', 'Lorcan', 'Saoirse'];

        this.basePrices = {
            iron: 20, copper: 40, steel: 60, amethyst: 100,
            emerald: 150, magmastone: 200, quartz: 300, titanium: 500
        };

        this.uiElements = [];
    }

    create() {
        // Request box and basic layout
        const panel = this.add.rectangle(960, 540, 600, 360, 0x222222, 0.95)
            .setStrokeStyle(4, 0xffffff, 0.5);

        const villagerName = this.villagerNames[this.buildingIndex % this.villagerNames.length];

        this.add.text(780, 420, `${villagerName}'s Request`, {
            fontSize: '32px', color: '#ffffff', fontFamily: 'Arial'
        });

        this.add.image(750, 480, `villager_${villagerName.toLowerCase()}`).setScale(1.5);
```

UIScene.js

Upgrade tree with dependencies, unlock progression, and game ending trigger.

```javascript
export default class UIScene extends Phaser.Scene {
    constructor() {
        super('UIScene');
    }

    create() {
        // Semi-transparent background covering whole screen
        this.background = this.add.rectangle(960, 540, 1920, 1080, 0x000000, 0.5);

        const menuWidth = 700;
        const menuHeight = 500;

        // Main upgrade menu background
        this.menu = this.add.rectangle(960, 540, menuWidth, menuHeight, 0x333333, 0.95)
            .setStrokeStyle(4, 0xffffff, 0.8);

        // Menu Title
        this.add.text(960, 340, 'Upgrade Tree', {
            fontSize: '36px',
            color: '#ffffff',
            fontFamily: 'Arial'
        }).setOrigin(0.5);

        // Upgrade list with prerequisites
        this.upgrades = [
            { name: 'Faster Production', price: 500, unlocked: false, prerequisite: null },
            { name: 'Extra Storage', price: 1000, unlocked: false, prerequisite: 0 },
            { name: 'Double Income', price: 2000, unlocked: false, prerequisite: 1 },
            { name: 'End of Game', price: 7000, unlocked: false, prerequisite: 2 }
        ];

        // Load unlocked upgrades and player money
        this.registryUpgrades = this.registry.get('unlockedUpgrades') || [];
        this.money = this.registry.get('money') || 0;

        this.drawUpgrades();

        // Money display
        this.moneyText = this.add.text(960, 310, `${this.money}`, {
            fontSize: '28px',
```